Introdução a Problemas NP
Verificação Eficiente e a Questão P vs NP

Pedro Ximenes

Universidade Católica de Pernambuco (UNICAP)

Análise de Algoritmos – 2026.1

Roteiro da Aula

Revisão – Aula 7: O que é a Classe P?

Ideia Central

Na Aula 7 medimos a complexidade de **problemas**, não só de algoritmos. Um problema pertence à classe P se existe algum algoritmo que o resolve em **tempo polinomial**.

Definição Formal

$$P = \left\{ \pi \text{ (decisão)} \;\middle|\; \exists \text{ algoritmo que resolve } \pi \text{ em } O(n^k),\ \exists k \right\}$$

Conceitos-Chave Revisados

- **Problema de decisão:** resposta é SIM ou NÃO.
- **Polinomial:** $O(n)$, $O(n^2)$, $O(n^3)$, ... são polinomiais.
- **Tratável:** problema que admite algoritmo polinomial.
- **Intratável:** problema onde nenhum algoritmo polinomial é conhecido.

Exemplo – IS-SORTED $\in P$

Array $A[0..n-1]$ está em ordem não-decrescente? Basta um scan linear $O(n) \Rightarrow$ IS-SORTED $\in P$.

Revisão – Exemplos de Problemas em P

Problemas Vistos na Aula 7

Problema	**Algoritmo**	**Complexidade**
IS-SORTED	Scan linear	$O(n)$
PALÍNDROMO	Dois ponteiros	$O(n)$
BUSCA-LINEAR	Scan linear	$O(n)$
CAMINHO (BFS)	Busca em largura	$O(\|V\| + \|E\|)$
MST (Kruskal)	Ordenar + Union-Find	$O(\|E\| \log \|V\|)$
PRIMALIDADE	AKS (2002)	$O(\log^{12} n)$

Ponte para a Aula 8

Todos esses problemas têm algoritmos polinomiais **conhecidos**.
Hoje estudaremos problemas onde **ninguém sabe** se existe algoritmo polinomial, mas onde **verificar** uma solução pronta é fácil.

Revisão – Por que Polinomial Importa?

Crescimento Comparativo

Complexidade	$n = 30$	$n = 60$	$n = 100$
$O(n^2)$	900	3 600	10 000
$O(n^3)$	27 000	216 000	10^6
$O(2^n)$	$\approx 10^9$	$\approx 10^{18}$	$\approx 10^{30}$
$O(n!)$	$\approx 10^{32}$	impossível	impossível

Consequência Prática

Para $n = 60$, um algoritmo $O(n^3)$ leva 216 000 operações (microssegundos a 1 GHz). Um algoritmo $O(2^n)$ levaria $\approx 10^{18}$ operações — **bilhões de anos**. A diferença é astronômica!

Mensagem

Algoritmos polinomiais são *viáveis* na prática. Algoritmos exponenciais são *inviáveis* mesmo para n modesto.

A Diferença Entre Resolver e Verificar

Situação Comum no Dia a Dia

- **Resolver** um Sudoku difícil: pode levar horas.
- **Verificar** se uma grade preenchida é solução válida: leva poucos segundos (basta conferir linhas, colunas e blocos 3×3).

A Assimetria Fundamental

Verificar: **FÁCIL** --- geralmente mais fácil que ---> Resolver: **DIFÍCIL?**

Exemplos Imediatos

Problema	**Resolver**	**Verificar**
Mochila 0-1	Selecionar itens (difícil)	Conferir peso e valor (rápido)
Sudoku	Preencher grade (difícil)	Conferir regras (rápido)
Soma-Subconj.	Encontrar subconj. (difícil)	Somar elem. (rápido)

Motivação – Mochila 0-1 (Enunciado)

O Problema da Mochila 0-1

Dado: n itens com pesos w_i e valores v_i; capacidade C e valor mínimo V.
Decisão: existe $T \subseteq \{1, \ldots, n\}$ tal que $\sum_{i \in T} w_i \leq C$ e $\sum_{i \in T} v_i \geq V$?

Por que é difícil resolver? (força bruta)

Há 2^n subconjuntos possíveis. Testar todos: $O(2^n \cdot n)$ — **exponencial**.

- $n = 20$: $2^{20} \approx 10^6$ subconjuntos. Tolerável.
- $n = 60$: $2^{60} \approx 10^{18}$ subconjuntos — **inviável!**
- $n = 100$: $2^{100} \approx 10^{30}$ subconjuntos — **impossível!**

Por que isso importa?

A Mochila 0-1 modela seleção de investimentos, alocação de recursos e corte de materiais. Encontrar a solução ótima é difícil — mas verificar uma proposta é fácil.

Motivação – Mochila 0-1 (Verificação)

Verificar é fácil!

Itens: $(w_1{=}3, v_1{=}4)$, $(w_2{=}5, v_2{=}6)$, $(w_3{=}2, v_3{=}3)$, $(w_4{=}4, v_4{=}5)$; $C{=}9$, $V{=}12$.
Certificado proposto: $T = \{1, 3, 4\}$ (itens 1, 3 e 4 selecionados).

Verificação Passo a Passo

1. $T \subseteq \{1, 2, 3, 4\}$? Sim. **OK**.
2. Peso total: $w_1 + w_3 + w_4 = 3 + 2 + 4 = 9 \leq C = 9$. **OK**.
3. Valor total: $v_1 + v_3 + v_4 = 4 + 3 + 5 = 12 \geq V = 12$. **OK**.

Certificado **VÁLIDO**. Custo total: $O(|T|) = O(n)$.

A Assimetria

- **Encontrar V:** $O(2^n \cdot n)$ por força bruta — **exponencial**.
- **Verificar V:** $O(n)$ — **linear, polinomial**.

Verificar é **exponencialmente mais fácil** que resolver!

Motivação – Soma de Subconjunto (Enunciado)

O Problema

Dado: conjunto $S = \{a_1, \ldots, a_n\}$ de inteiros positivos e alvo k.
Decisão: existe $T \subseteq S$ com $\sum_{a \in T} a = k$?

Por que é difícil resolver? (força bruta)

Um conjunto com n elementos tem 2^n subconjuntos. Testar todos: $O(2^n \cdot n)$ — **exponencial**.

- $n = 30$: $2^{30} \approx 10^9$ subconjuntos.
- $n = 60$: $2^{60} \approx 10^{18}$ subconjuntos — inviável!

Exemplo Concreto

$S = \{3, 5, 7, 11, 13\}, \quad k = 25.$

Busca por força bruta (alguns subconjuntos):
$\{3, 5\} \to 8$, $\{3, 7\} \to 10$, $\{5, 7\} \to 12$, $\{3, 11\} \to 14$, $\{3, 5, 7\} \to 15$, $\{5, 7, 13\} \to$ **25 ACHOU!**

Motivação – Soma de Subconjunto (Verificação)

Verificar é fácil!

Instância: $S = \{3, 5, 7, 11, 13\}, \quad k = 25.$
Certificado proposto: $T = \{5, 7, 13\}.$

Verificação Passo a Passo

1. $T \subseteq S$? — $5 \in S$? SIM. $7 \in S$? SIM. $13 \in S$? SIM. **OK**.
2. Elementos distintos em V? $\{5, 7, 13\}$ — sem repetição. **OK**.
3. Calcule $\sum T = 5 + 7 + 13 = 25$. **OK**.
4. $\sum T = k$? $25 = 25$? **SIM**.

Certificado **VÁLIDO**. Resposta: **SIM**.
Custo: $O(|T|) = O(n)$ — linear, polinomial!

Conclusão

Encontrar o subconjunto: $O(2^n)$ no pior caso. Verificar o subconjunto: $O(n)$.
Verificação é exponencialmente mais rápida que resolução!

O que é um Certificado?

Definição Informal

Um **certificado** (ou **testemunha**) é uma informação adicional que **prova** que a resposta de um problema é "SIM", e que pode ser **verificada rapidamente**.

Três Requisitos do Certificado

1. **Curto:** tamanho polinomial no tamanho da entrada ($|c| = O(|x|^k)$).
2. **Verificável:** existe algoritmo que verifica c em tempo polinomial.
3. **Suficiente:** se a resposta é SIM, *existe* um certificado válido.

Analogia Legal

Em um tribunal:

- **Problema:** "Esse acusado é inocente?"
- **Certificado de inocência:** álibi, vídeo, testemunha.
- **Verificação:** o juiz analisa a prova rapidamente.
- **Sem prova:** pode ser impossível provar inocência!

Certificados – Exemplos

Tabela de Certificados por Problema

Problema	**Certificado**	**Tamanho**	**Verificação**
MOCHILA 0-1	Lista de índices V	$O(n)$	$O(n)$
SAT	Atribuição das variáveis	$O(n)$	$O(m \cdot n)$
SUDOKU	Grade 9×9 completa	$O(1)$	$O(1)$
SOMA-SUBCONJ.	Lista de índices V	$O(n)$	$O(n)$
CLIQUE	Conjunto C de k vértices	$O(k)$	$O(k^2)$
CICLO-HAM.	Sequência de vértices	$O(n)$	$O(n)$

Observação Importante

Todos os certificados têm tamanho **polinomial** e verificação **polinomial**. Isso é o que os torna certificados válidos para NP.

O que acontece quando a resposta é NÃO?

Para instâncias "NÃO", *não existe* certificado que convença o verificador. Ex.: $S = \{2, 4, 6\}$, $k = 11$. Nenhum subconjunto soma 11 — nenhum certificado é válido.

O Algoritmo Verificador – Estrutura Geral

Como Funciona um Verificador

O verificador V recebe **dois inputs**:

- x: a instância do problema (a pergunta original).
- c: o certificado proposto (a prova de que a resposta é SIM).

E retorna **SIM** se c prova que $\pi(x) = \text{SIM}$, e **NÃO** caso contrário.

Verificador para MOCHILA 0-1 e SAT

MOCHILA 0-1: $(T, C, V) \rightarrow$ some pesos e valores dos itens em V; retorne SIM se $\sum w \leq C$ e $\sum v \geq V$. Custo: $O(n)$.
SAT: $(\phi, \text{atrib.}) \rightarrow$ substitua valores em cada cláusula; retorne SIM se todas V. Custo: $O(m \cdot n)$.

Definição Formal da Classe NP

Definição via Verificador Polinomial

A classe **NP** *(Nondeterministic Polynomial time)* é o conjunto de todos os problemas de decisão π para os quais existe um **verificador polinomial** V tal que:

1. Se $\pi(x) = \text{SIM}$, *existe* certificado c com $|c| = O(|x|^k)$ e $V(x, c) = \text{SIM}$.
2. Se $\pi(x) = \text{NÃO}$, *para todo* c, $V(x, c) = \text{NÃO}$.
3. $V(x, c)$ roda em $O(|x|^j)$ para algum j (polinomial).

Em Notação Compacta

$$NP = \{\pi \mid \exists V \text{ polinomial} : \pi(x)=\text{SIM} \Leftrightarrow \exists c, |c| \leq |x|^k, V(x, c)=\text{SIM}\}$$

O Que Significa NP?

N = *Nondeterministic*: oráculo mágico que advinha o certificado correto.
P = *Polynomial*: o verificador roda em tempo polinomial.
Juntos: "resolução mágica + verificação eficiente".

NP em Linguagem Acessível

Dito de Outra Forma

$\pi \in NP$ significa que:

- Para cada instância com resposta **SIM**: existe uma **prova curta** que qualquer verificador polinomial aceita.
- Para instâncias com resposta **NÃO**: nenhuma prova curta pode enganar o verificador.

Equívoco Comum

ERRADO: "NP significa difícil de resolver."
CERTO: "NP significa fácil de *verificar*."

Problemas em P **também** estão em NP!

Exemplos Imediatos de NP

- IS-SORTED $\in P \subseteq NP$ (resolve e verifica em $O(n)$).
- SAT $\in NP$ (verifica em $O(m \cdot n)$, resolver parece $O(2^n)$).
- SUDOKU $\in NP$ (verifica em $O(1)$, resolver pode ser exponencial).
- CICLO-HAMILTONIANO $\in NP$ (verifica em $O(n)$, resolver é $(n-1)!$ no pior caso).

O Que Significa "Não Determinístico"?

Máquina de Turing Não Determinística

A cada passo, a máquina pode **bifurcar** em múltiplos caminhos simultaneamente. Se *algum caminho* retorna SIM, ela retorna SIM.

Determinístico (P?)

Testa candidatos **um a um**:

- $c_1 \rightarrow$ NÃO
- $c_2 \rightarrow$ NÃO
- $\vdots$
- $c_k \rightarrow$ **SIM!**

Custo: $O(2^n)$ no pior caso.

Não Determinístico (NP)

"**Adivinha**" o candidato certo:

- $c^* \rightarrow$ **SIM!**

Custo: $O(n^k)$ — polinomial.

Intuição Prática

Um oráculo mágico **adivinha** a solução. Se ela puder ser **verificada em tempo polinomial**, o problema está em NP.

Analogia: A Coleção de Chaves

Cenário

Você tem uma **fechadura** e $N = 2^n$ chaves. Qual abre?

Determinístico (P?)

1. Pega chave 1. Não abre.
2. Pega chave 2. Não abre.

$\vdots$

k Pega chave k. **Abre!**

Custo: até 2^n tentativas.

Não Determinístico (NP)

1. **Adivinha** a chave k^*.
2. Testa: k^* abre? **Sim!**

Custo: $O(n^k)$ para verificar.

O Paralelo com NP

Chave k^*	$\longleftrightarrow$	Certificado c
Testar se abre	$\longleftrightarrow$	Verificador $V(x, c)$
Adivinhar	$\longleftrightarrow$	Não determinismo

Resumo — O Que é NP?

Conceito	**Significado**
NP	Verificável em tempo polinomial
Certificado c	Prova curta ($O(\|x\|^k)$) da resposta SIM
Verificador V	Algoritmo polinomial que confere o certificado
Não-determinístico	"Adivinha" o certificado instantaneamente
$P \subseteq NP$	Resolver em poli. $\Rightarrow$ verificar em poli.

A Mensagem-Chave

NP não significa "difícil de resolver".

NP significa **"fácil de verificar"**.

A Grande Questão em Aberto

Todo problema fácil de verificar também é fácil de resolver? Ou seja: $P = NP$?

Esta é a maior questão em aberto da Ciência da Computação.

Exemplo 1 – SAT: O Problema (Enunciado)

Problema de Decisão: SAT

SAT (Boolean Satisfiability): Dada uma fórmula booleana ϕ em **Forma Conjuntiva Normal (CNF)**, decidir se existe uma atribuição de verdade às variáveis que torna ϕ verdadeira.

Vocabulário Necessário

- **Variável booleana:** x, y, z, ... tomam valores em $\{V, F\}$ (verdadeiro/falso).
- **Literal:** variável (x) ou sua negação ($\neg x$).
- **Cláusula:** disjunção (`OR`) de literais. Ex.: $(x \vee \neg y \vee z)$.
- **Fórmula CNF:** conjunção (`AND`) de cláusulas.

Exemplo de Fórmula CNF

$$\phi = (x \vee y \vee \neg z) \ \wedge \ (\neg x \vee y) \ \wedge \ (\neg y \vee z)$$

- 3 variáveis: x, y, z.
- 3 cláusulas.
- **Pergunta:** existe atribuição que torna ϕ verdadeira?

Exemplo 1 – SAT: Por que é Difícil Resolver?

Força Bruta

Com n variáveis, há 2^n atribuições possíveis. Para cada uma, é preciso avaliar as m cláusulas com n literais. Custo total: $O(2^n \cdot m \cdot n)$ — **exponencial**.

Exemplo: $n = 3$ variáveis, $\phi = (x \vee y) \wedge (\neg x \vee z) \wedge (\neg y \vee \neg z)$

x	y	z	Cl.1	Cl.2	Cl.3	ϕ
F	F	F	F	V	V	F
F	F	V	F	V	V	F
F	V	F	V	V	F	F
F	V	V	V	V	F	F
V	F	F	V	F	V	F
V	F	V	V	V	V	**V**
V	V	F	V	F	F	F
V	V	V	V	V	F	F

Apenas $(x{=}V, y{=}F, z{=}V)$ satisfaz ϕ. Testamos 8 combinações. Para $n = 60$: $2^{60} \approx 10^{18}$ combinações — **inviável**!

Exemplo 1 – SAT: Certificado e Verificador (Enunciado)

Tarefa

Dada a fórmula $\phi = (x \vee y \vee \neg z) \wedge (\neg x \vee y) \wedge (\neg y \vee z)$ com certificado $c = (x{=}V,\ y{=}V,\ z{=}V)$:

(a) O que é o certificado para SAT?

(b) Verifique se c é válido, cláusula por cláusula.

(c) Qual o custo do verificador?

Contexto

Lembre-se: V significa Verdadeiro e F significa Falso. Uma cláusula é verdadeira se *pelo menos um* literal dela é verdadeiro. A fórmula CNF é verdadeira se *todas* as cláusulas são verdadeiras.

Exemplo 1 – SAT: Certificado e Verificador (Resolução)

(a) O certificado para SAT

O certificado é a **atribuição de verdade** para todas as variáveis. Para n variáveis: tamanho $O(n)$ — polinomial.

(b) Verificação cláusula por cláusula — $c = (x{=}V,\ y{=}V,\ z{=}V)$

1. $(x \vee y \vee \neg z) = (V \vee V \vee F) = V$ **OK**
2. $(\neg x \vee y) = (F \vee V) = V$ **OK**
3. $(\neg y \vee z) = (F \vee V) = V$ **OK**

Todas as cláusulas verdadeiras $\Rightarrow$ **certificado VÁLIDO**. A fórmula é satisfazível com (V, V, V).

(c) Custo do verificador

Para cada cláusula (m total), avalia no máximo n literais.
Custo total: $O(m \cdot n)$ — **polinomial**. Portanto SAT $\in NP$. ■

SAT $\in P$?

Ninguém sabe! SAT é o **primeiro problema NP-completo** (Cook, 1971).

Exemplo 1 – SAT: Algoritmos Modernos

Sem Algoritmo Polinomial Conhecido

Apesar de SAT estar em NP, nenhum algoritmo polinomial foi encontrado para o caso geral. Algoritmos práticos exploram estrutura e heurísticas.

Algoritmos SAT (SAT Solvers)

DPLL (Davis-Putnam, 1962): propagação de unidade + backtracking.
CDCL (Conflict-Driven Clause Learning): aprende cláusulas de conflitos; poda inteligente do espaço de busca.

Aplicações de SAT Solvers

- Verificação formal de hardware (Intel, AMD).
- Compiladores, otimizadores e planejamento em IA.
- CDCL resolve instâncias industriais com 10^6 variáveis.
- Sem garantia polinomial, mas eficientes na prática!

Exemplo 2 – SUDOKU: O Problema (Enunciado)

Problema de Decisão: SUDOKU

SUDOKU: Dada uma grade 9×9 parcialmente preenchida com dígitos 1–9, decidir se existe um **preenchimento válido** completo.

Regras do Preenchimento Válido

1. **Linha:** cada linha deve conter os dígitos 1–9 exatamente uma vez.
2. **Coluna:** cada coluna deve conter os dígitos 1–9 exatamente uma vez.
3. **Bloco:** cada bloco 3×3 deve conter os dígitos 1–9 exatamente uma vez.

Espaço de Busca

Uma grade 9×9 tem 81 células. Número de formas de preencher (sem restrições): $9^{81} \approx 2 \times 10^{77}$. Com restrições, o número diminui, mas ainda é enorme.

Resultado Teórico

SUDOKU generalizado ($n^2 \times n^2$ com n^2 símbolos) é **NP-completo**.

Exemplo 2 – SUDOKU: Instância e Verificação

5	3			7				
6			1	9	5			
	9	8					6	
8				6				3
4			8					1
	7				2			6
			4	1	9			
						5	8	

Puzzle parcial

Certificado

A grade 9×9 completamente preenchida.
Tamanho: 81 células $= O(1)$ (constante).

Verificação (Custo $O(1)$)

- 9 linhas: contem $\{1, \ldots, 9\}$? ($9 \times 9 = 81$ cheques)
- 9 colunas: idem. (81 cheques)
- 9 blocos 3×3: idem. (81 cheques)
- Total: 243 cheques — constante!

SUDOKU $\in NP$

Certificado $O(1)$ + verificação $O(1) \Rightarrow$ SUDOKU $\in NP$. ■

Exemplo 3 – SOMA-SUBCONJUNTO: Enunciado Formal

Problema de Decisão: SOMA-SUBCONJUNTO

Entrada: conjunto $S = \{a_1, a_2, \ldots, a_n\}$ de inteiros positivos e alvo k.
Saída: existe subconjunto $T \subseteq S$ tal que $\sum_{a \in T} a = k$?

Importância Prática

- Caso especial do **Problema da Mochila 0-1** (peso = valor, capacidade = k).
- Aparece em criptografia, planejamento financeiro e corte de materiais.

Instâncias de Exemplo

SIM: $S = \{4, 7, 2, 9, 1, 6\}$, $k = 16$. Existe $T = \{7, 9\}$: $7 + 9 = 16$. **SIM.**
NÃO: $S = \{2, 4, 6\}$, $k = 11$. Somas possíveis: 0,2,4,6,8,10,12. Nenhuma $= 11$. **NÃO.**

Exemplo 3 – SOMA-SUBCONJUNTO: SOMA-SUBCONJUNTO $\in NP$

Certificado

O próprio subconjunto V: lista dos elementos escolhidos.
Tamanho: $|T| \leq n$ — **polinomial**.

Verificação Passo a Passo — S=$\{4, 7, 2, 9, 1, 6\}$, k=16, T=$\{7, 9\}$

1. $T \subseteq S$? SIM. $7 \in S$? SIM. $9 \in S$? SIM. **OK**.
2. Elementos distintos em V? $\{7, 9\}$ — sem repetição. **OK**.
3. Soma $= 7 + 9 = 16$. **OK**.
4. $16 = k = 16$? **SIM**.

Certificado VÁLIDO. Custo: $O(n)$ — polinomial. Portanto SOMA-SUBCONJ. $\in NP$.
■

SOMA-SUBCONJUNTO $\in P$?

Existe solução via Programação Dinâmica em $O(n \cdot k)$. Mas isso é **pseudo-polinomial**: se $k = 2^n$, o custo é exponencial. Para o caso geral (k arbitrário), SOMA-SUBCONJUNTO é NP-completo.

Exemplo 3 – SOMA-SUBCONJUNTO: Solução por DP

Programação Dinâmica

$dp[i][s]{=}V$ se existe subconj. dos i primeiros que soma s.
Base: $dp[0][0]{=}V$; $dp[0][s]{=}F$ para $s > 0$.
Transição: $dp[i][s] = dp[i{-}1][s] \vee (s{\geq}a_i \wedge dp[i{-}1][s{-}a_i])$
Resposta: $dp[n][k]$.

Tabela DP — $S = \{2, 3, 7\}$, $k = 5$

	$s{=}0$	$s{=}1$	$s{=}2$	$s{=}3$	$s{=}4$	$s{=}5$
$i{=}0$ (vazio)	V	F	F	F	F	F
$i{=}1$ ($a_1{=}2$)	V	F	**V**	F	F	F
$i{=}2$ ($a_2{=}3$)	V	F	V	**V**	F	**V**
$i{=}3$ ($a_3{=}7$)	V	F	V	V	F	V

$dp[3][5] = V \Rightarrow$ existe subconjunto $\{2, 3\}$ que soma 5.

Complexidade da DP

$O(n \cdot k)$ — pseudo-polinomial. Se $k = 2^n$: exponencial. Não é polinomial no caso geral.

Exemplo 4 – CICLO-HAMILTONIANO: O Problema (Enunciado)

Definição: Ciclo Hamiltoniano

Um **ciclo hamiltoniano** em um grafo $G = (V, E)$ é um ciclo que:

1. Passa por **cada vértice exatamente uma vez**.
2. Retorna ao vértice de início.

É a diferença entre um ciclo comum (pode repetir vértices) e um hamiltoniano (todos visitados uma vez).

Problema de Decisão: CICLO-HAMILTONIANO

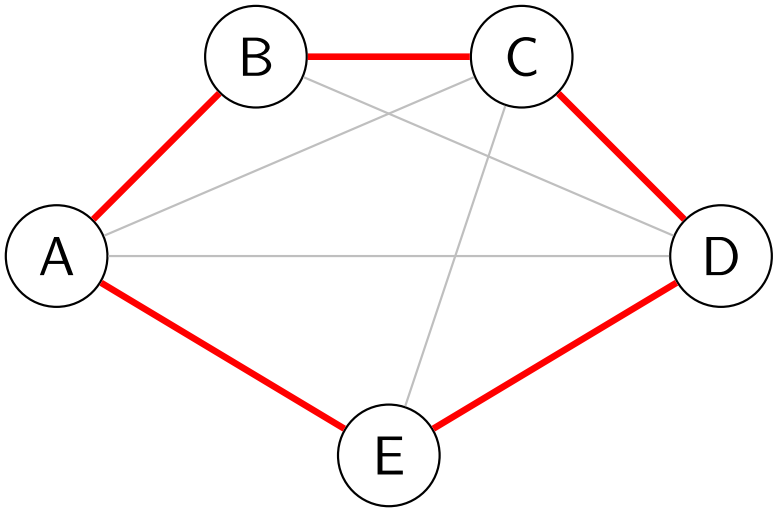
Entrada: grafo não-dirigido $G = (V, E)$.
Saída: G possui ciclo hamiltoniano?

Por que é Difícil?

Com n vértices, há $(n-1)!$ possíveis sequências de visita.

- $n = 10$: $(9)! = 362\,880$. Tolerável.
- $n = 20$: $(19)! \approx 1{,}2 \times 10^{17}$. **Inviável!**
- $n = 30$: $(29)! \approx 8{,}8 \times 10^{30}$. **Impossível!**

Exemplo 4 – CICLO-HAM.: Grafo com Ciclo Hamiltoniano



Ciclo: A→B→C→D→E→A

Verificação do Ciclo (A,B,C,D,E)

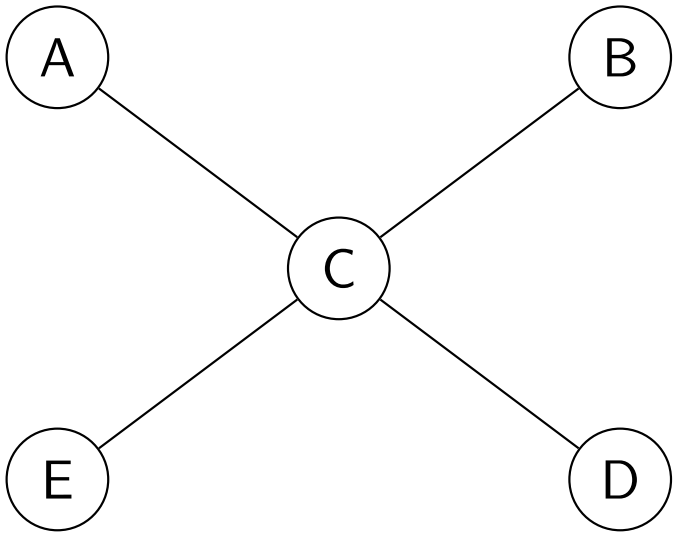
1. Cada vértice aparece uma vez? A,B,C,D,E — todos distintos. **OK**.
2. Arestas existem? AB? sim. BC? sim. CD? sim. DE? sim. EA? sim. **OK**.

Custo: $O(n)$ — polinomial.

Conclusão

Certificado = sequência de vértices ($O(n)$).
Verificação = $O(n)$.
CICLO-HAM. $\in NP$. ■

Exemplo 4 – CICLO-HAM.: Grafo sem Ciclo Hamiltoniano



Estrela: só C tem grau 4

Por que não existe ciclo hamiltoniano?

Para ter ciclo hamiltoniano, o grafo precisa ter um ciclo que visita A, B, C, D, E uma vez. Após entrar em C, precisamos sair por outra aresta. Mas C só se conecta a A, B, D, E — não há aresta entre A, B, D, E. Portanto, qualquer caminho que chega a uma "folha" (A, B, D ou E) não consegue continuar. Impossível!

Conclusão

Este grafo **não tem** ciclo hamiltoniano. Resposta: **NÃO**. Nenhum certificado pode convencer o verificador.

Exemplo 4 – CICLO-HAM.: Conexão com TSP

Ciclo Hamiltoniano e o Caixeiro Viajante

O **Problema do Caixeiro Viajante (TSP)** generaliza CICLO-HAMILTONIANO:

- **CICLO-HAMILTONIANO:** grafo simples, cada aresta tem peso 1; existe tour de custo $= n$?
- **TSP:** grafo completo com pesos; existe tour com custo $\leq L$?

CICLO-HAMILTONIANO é um caso especial de TSP!

Importância Prática do TSP

- Logística: roteirização de caminhões, entregadores.
- Produção: montagem de circuitos impressos (PCB drilling).
- Genética: sequenciamento de DNA.

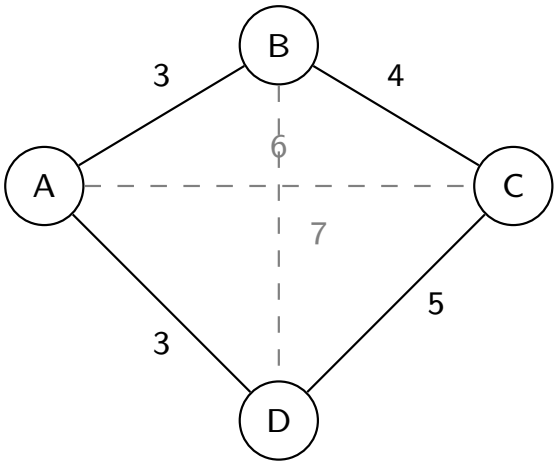
CICLO-HAM. e TSP em NP?

Ambos em NP: certificado = tour (sequência de vértices, tamanho $O(n)$); verificação = checar arestas e somar pesos, $O(n)$ — polinomial. Ambos são NP-completos (Karp, 1972).

Exemplo 5 – TSP: Enunciado e Instância

Problema de Decisão: TSP

Entrada: grafo completo com pesos nas arestas e limite L.
Saída: existe **tour hamiltoniano** com custo total $\leq L$?
(Tour hamiltoniano = ciclo que visita cada vértice exatamente uma vez.)



Instância

$L = 15$.
Tour: $A \to B \to C \to D \to A$
Custo: $3 + 4 + 5 + 3 = 15 \leq L$.
Resposta: SIM.

Certificado

O tour (sequência de vértices).
Tamanho: $O(n)$ — polinomial.

TSP $\in NP$

Verificação em $O(n)$. ■

Teorema: $P \subseteq NP$ (Enunciado)

Teorema

Todo problema de decisão em P também está em NP.
Em notação: $P \subseteq NP$.

Intuição

Se sabemos **resolver** um problema em tempo polinomial, então certamente sabemos **verificar** uma solução proposta em tempo polinomial: basta resolver o problema e comparar.

O que Precisa Ser Demonstrado

Para cada $\pi \in P$, precisamos exibir:

1. Um **certificado** de tamanho polinomial.
2. Um **verificador polinomial** que usa esse certificado.

A demonstração deve funcionar para **qualquer** $\pi \in P$.

Teorema: $P \subseteq NP$ (Prova Completa)

Prova — Passo a Passo

Hipótese: $\pi \in P$.
Logo, existe algoritmo A que resolve π em $O(n^k)$ para algum k.

Construção do verificador V:
Dado input (x, c): *ignore o certificado* c e execute $A(x)$.

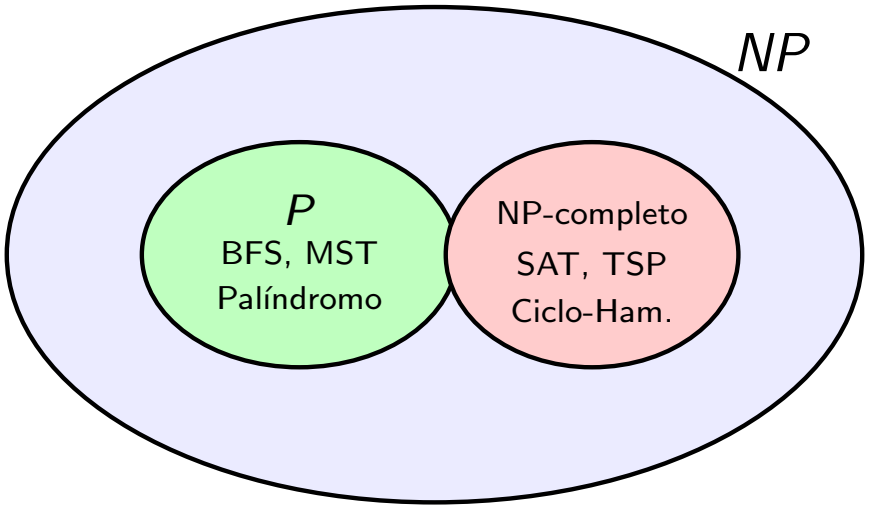
- Se $A(x) = \text{SIM} \Rightarrow V(x, c) = \text{SIM}$.
- Se $A(x) = \text{NÃO} \Rightarrow V(x, c) = \text{NÃO}$.

Verificação das três propriedades de NP:

1. **Completude:** se $\pi(x) = \text{SIM}$, então $A(x) = \text{SIM}$, então $V(x, c) = \text{SIM}$ para qualquer c. **OK.**
2. **Solidez:** se $\pi(x) = \text{NÃO}$, então $A(x) = \text{NÃO}$, então $V(x, c) = \text{NÃO}$ para todo c. **OK.**
3. **Eficiência:** V roda em $O(n^k)$ (mesmo custo de A) — polinomial. **OK.**

V é verificador polinomial $\Rightarrow \pi \in NP$. ■

Diagrama de Classes de Complexidade



O que está provado

$$P \subseteq NP \quad \text{(comprovado)}$$

A inclusão estrita $P \subsetneq NP$ é fortemente acreditada, mas **não provada**.
Se $P = NP$: todos os NP seriam resolúveis em tempo polinomial (improvável).

A Grande Pergunta: $P = NP$?

Formulação da Questão

"Se verificar uma solução é fácil (tempo polinomial),
isso implica que **encontrar** a solução também é fácil?"

Dois Cenários

1. $P = NP$: toda vez que verificação é fácil, existe algoritmo polinomial para resolução. *(Seria revolucionário — e provavelmente destrutivo para a criptografia!)*
2. $P \neq NP$: existem problemas onde verificação é fácil mas resolução é exponencialmente difícil. *(O que a maioria dos pesquisadores acredita.)*

Prêmio Milênio

O Clay Mathematics Institute oferece **$1.000.000** para quem provar ou refutar $P = NP$. É um dos 7 Problemas do Milênio.

Implicações de $P = NP$

Se $P = NP$ (cenário hipotético)

- **Criptografia comprometida:** RSA, AES e outros sistemas pressupõem $P \neq NP$. Se $P = NP$, fatoração e logaritmo discreto seriam fáceis — criptografia quebraria.
- **Otimização eficiente:** TSP, Mochila 0-1, Roteamento — todos resolúveis em tempo polinomial.
- **Inteligência Artificial:** planejamento, scheduling, aprendizado — muito mais eficientes.
- **Ciência:** provar teoremas seria automatizável em tempo polinomial.

Se $P \neq NP$ (cenário acreditado)

- A criptografia moderna tem fundamentos sólidos.
- Existem problemas que são fundamentalmente difíceis de resolver, por mais inteligente que seja o algoritmo.
- A fronteira entre "fácil" e "difícil" é real e permanente.

Por que a Maioria Acredita que $P \neq NP$?

Argumento 1 – Evidência Empírica (50+ anos)

Desde 1971, milhares de pesquisadores tentaram encontrar algoritmos polinomiais para SAT, TSP, CICLO-HAMILTONIANO, etc. Nenhum foi encontrado. Se $P = NP$, tais algoritmos existem — mas nenhum foi descoberto em mais de 50 anos de esforço intensivo.

Argumento 2 – Intuição Filosófica

"Verificar" parece fundamentalmente mais fácil que "criar". É a diferença entre reconhecer uma boa solução e descobri-la do zero.

Analogia do Labirinto

Dado o labirinto resolvido (caminho marcado), é imediato verificar que é correto. Mas *encontrar* o caminho pode ser muito difícil.

Situação Atual

Provado: $P \subseteq NP$.
Em aberto: $P = NP$ ou $P \neq NP$?
Consenso: a maioria acredita $P \neq NP$, mas sem prova formal.

Linha do Tempo Histórica

Marcos em Complexidade Computacional

Ano	Evento
1936	Turing: Problema da Parada; limites da computação.
1965	Cobham/Edmonds: P como "computacionalmente fácil".
1971	Cook: SAT é NP-completo (Teorema de Cook).
1972	Karp: 21 problemas NP-completos clássicos.
1973	Levin (URSS): independentemente prova SAT NP-completo.
1979	Garey & Johnson: "Computers and Intractability" (a bíblia).
2000	Clay Institute: \$1M para quem resolver P vs NP.
2010	Deolalikar propõe "prova" de $P \neq NP$ — invalidada.
Hoje	P vs NP: em aberto.

Garey & Johnson (1979)

Catalogaram centenas de problemas NP-completos. É a obra de referência para quem precisa identificar se um novo problema é NP-completo.

O que é NP-Completo?

Motivação

Dentro da classe NP, alguns problemas são "os mais difíceis". Se conseguirmos resolver qualquer um deles em tempo polinomial, **todos** os problemas em NP poderiam ser resolvidos em tempo polinomial — e $P = NP$.

Definição Informal

Um problema $\pi \in NP$ é **NP-completo** se:

1. $\pi \in NP$ (verificável em tempo polinomial).
2. Todo problema em NP pode ser **reduzido** a π em tempo polinomial ("π é pelo menos tão difícil quanto qualquer problema em NP").

O que Significa "Redução Polinomial" de A para B?

Transformar qualquer instância de A em uma instância equivalente de B em tempo polinomial. Se B está em P, então A também está!

O Teorema de Cook-Levin (1971)

O Primeiro Problema NP-Completo

Teorema de Cook (1971) / Levin (1973): SAT é NP-completo. Todo problema em NP pode ser reduzido a SAT em tempo polinomial.

Consequência Imediata

- Se SAT $\in P \Rightarrow P = NP$ (todos os problemas NP seriam em P).
- Como SAT parece não estar em P, temos forte evidência de $P \neq NP$.
- SAT é o "calibrador de dificuldade" para todos os problemas em NP.

Como Provar que Novo Problema é NP-Completo

1. Mostre que $\pi \in NP$ (descreva certificado e verificador).
2. Reduza um NP-completo conhecido (ex.: SAT) a π em tempo polinomial.

Na Prática

Se um problema novo é NP-completo: provavelmente não existe algoritmo polinomial. Use aproximações, heurísticas ou restrinja-se a casos especiais.

Exemplos de Problemas NP-Completos

Tabela Clássica (Karp, 1972)

Problema	**Descrição Resumida**
SAT / 3-SAT	Satisfatibilidade booleana
CLIQUE	Existe clique de tamanho $\geq k$?
CONJUNTO INDEPENDENTE	Existe conj. independente de tamanho $\geq k$?
COBERTURA DE VÉRTICES	Existe cobertura com $\leq k$ vértices?
CICLO HAMILTONIANO	Existe ciclo que visita todos os vértices?
TSP (decisão)	Existe tour hamiltoniano com custo $\leq L$?
SOMA-SUBCONJUNTO	Existe subconjunto com soma $= k$?
MOCHILA 0-1	Existe seleção com valor $\geq V$ e peso $\leq W$?
COLORAÇÃO	O grafo é k-colorível?

Mensagem Prática

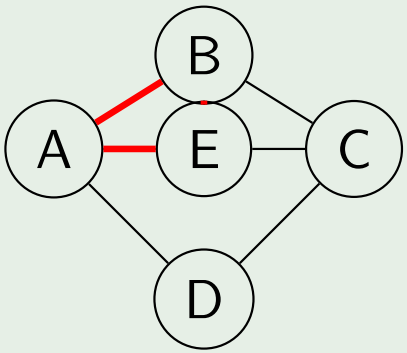
Ao identificar que seu problema é NP-completo: considere aproximação, heurísticas (genéticos, SA, GRASP), branch-and-bound, ou exploração de casos especiais em P.

Exercício 1 – CLIQUE (Enunciado)

Enunciado

O problema **CLIQUE**: dado grafo $G = (V, E)$ e inteiro k, decidir se G possui um clique (subgrafo completo) de tamanho $\geq k$.

Exemplo: $k = 3$



Clique $\{A, B, E\}$

Tarefa

Descreva o certificado e o verificador.

Mostre formalmente que CLIQUE $\in NP$, indicando:

(a) O certificado e seu tamanho.

(b) O algoritmo verificador.

(c) O custo do verificador.

Exercício 1 – CLIQUE (Resolução)

Passo 1 – Certificado

O certificado é o conjunto $C = \{v_1, v_2, \ldots, v_k\} \subseteq V$.
Tamanho: $|C| = k \leq |V| = n$ — **polinomial**.

Passo 2 – Verificador

Dado (G, k, C):

1. Verifique $|C| \geq k$. Custo: $O(k)$.
2. Para cada par $(u, v) \in C \times C$, $u \neq v$: verifique $(u, v) \in E$.
   Há $\binom{k}{2} \leq k^2/2$ pares. Custo: $O(k^2)$.

Total: $O(k^2) \leq O(n^2)$ — **polinomial**.

Passo 3 – Conclusão

Certificado de tamanho $O(n)$ + verificação em $O(n^2) \Rightarrow$ **CLIQUE** $\in NP$. ■

CLIQUE $\in P$?

CLIQUE é NP-completo (Karp, 1972). Provavelmente $\notin P$. O melhor algoritmo exato conhecido é $O(2^{0.249n})$ — ainda exponencial.

Exercício 2 – Verificar Certificado de SOMA-SUBCONJ. (Enunciado)

Enunciado

Dado:

- $S = \{4, 7, 2, 9, 1, 6\}$, alvo $k = 16$.
- Certificado proposto: $T = \{7, 9\}$.

Tarefas:

1. Execute o verificador passo a passo para V.
2. O certificado é válido?
3. Qual o custo do verificador para uma instância genérica?

Lembre-se: Verificador para SOMA-SUBCONJUNTO

O verificador checa:

1. Todos os elementos de V pertencem a S.
2. Não há repetições em V.
3. $\sum_{a \in T} a = k$.

Exercício 2 – Verificar Certificado de SOMA-SUBCONJ. (Resolução)

Verificação de $T=\{7,9\}$ sobre $S=\{4,7,2,9,1,6\}$, $k=16$

1. $T \subseteq S$? $7 \in S$? **SIM.** $9 \in S$? **SIM.** **OK.**
2. Elementos distintos? $\{7,9\}$ sem repetição. **OK.**
3. Soma: $7+9=16$. **OK.**
4. $16=k=16$? **SIM.**

Certificado $T=\{7,9\}$ é **VÁLIDO**. Resposta: **SIM**.

Conclusão

Custo genérico: $O(|T|)=O(n)$ — polinomial. Portanto SOMA-SUBCONJ. $\in NP$. ■

Exercício 3 – Verificar Certificado de SAT (Enunciado)

Enunciado

Fórmula:

$$\phi = (a \vee \neg b) \ \wedge \ (\neg a \vee c) \ \wedge \ (b \vee \neg c) \ \wedge \ (\neg a \vee \neg b \vee c)$$

Certificado proposto: $a = F$, $b = F$, $c = V$.

Tarefas:

1. Execute o verificador cláusula por cláusula.
2. O certificado é válido?
3. Se não é, isso significa que ϕ é insatisfazível?

Regra de Avaliação

Para avaliar uma cláusula: substitua cada variável pelo seu valor no certificado. $\neg V = F$ e $\neg F = V$. Uma cláusula é verdadeira se *ao menos um* literal é verdadeiro.

Exercício 3 – Verificar Certificado de SAT (Resolução)

Verificação — (a=F, b=F, c=V)

1. $(a \vee \neg b) = (F \vee \neg F) = (F \vee V) = V$ **OK**
2. $(\neg a \vee c) = (\neg F \vee V) = (V \vee V) = V$ **OK**
3. $(b \vee \neg c) = (F \vee \neg V) = (F \vee F) = F$ **FALHA!**

Cláusula 3 é falsa — o verificador retorna **NÃO**.

Conclusão

O certificado (a=F, b=F, c=V) é **INVÁLIDO**. Esta atribuição não satisfaz ϕ.

Isso Significa que ϕ é Insatisfazível?

NÃO! Apenas que *este* certificado não funciona. Outros certificados podem ser válidos. Por exemplo, tente (a=V, b=F, c=V):

1. $(V \vee V) = V$. **OK.**
2. $(F \vee V) = V$. **OK.**
3. $(F \vee F) = F$. **FALHA** novamente.

Fica como exercício: encontrar uma atribuição satisfatória para ϕ (se existir).

Exercício 4 – COLORAÇÃO $\in$ *NP* (Enunciado)

Enunciado

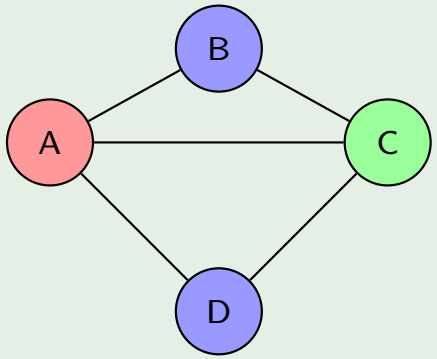
Dado grafo $G = (V, E)$ e inteiro k, G é **k-colorível**?
Isso significa: atribuir uma cor de $\{1, \ldots, k\}$ a cada vértice de modo que **vértices adjacentes tenham cores distintas**.

Tarefa

Descreva certificado e verificador.

Mostre que COLORAÇÃO $\in$ *NP*.

Exemplo: $k = 3$



cores: VM, AZ, VD

Exercício 4 – COLORAÇÃO $\in$ *NP* (Resolução)

Passo 1 – Certificado

Função $f : V \to \{1, \ldots, k\}$ que atribui uma cor a cada vértice.
Tamanho: $|f| = |V| = n$ — **polinomial**.

Passo 2 – Verificador: dado (G, k, f)

1. Para cada $v \in V$: verificar $1 \leq f(v) \leq k$. Custo: $O(|V|)$.
2. Para cada $(u, v) \in E$: verificar $f(u) \neq f(v)$. Custo: $O(|E|)$.

Total: $O(|V| + |E|)$ — **polinomial**.

Passo 3 – Conclusão

Certificado $O(n)$ + verificação $O(n + m) \Rightarrow$ **COLORAÇÃO** $\in$ *NP*. ■

Importância e Casos Especiais

Aplicações: frequências de rádio, escalonamento de tarefas.
NP-completo para $k \geq 3$; para $k = 2$ (IS-BIPARTITE) está em P (BFS).

Exercício 5 – IS-BIPARTITE: $P \subseteq NP$ (Enunciado)

Enunciado

O problema **IS-BIPARTITE** é:

- **Entrada:** grafo não-dirigido $G = (V, E)$.
- **Saída:** G é bipartido (os vértices podem ser divididos em dois grupos L e R sem arestas internas)?

Dado: IS-BIPARTITE $\in P$ (BFS em $O(|V| + |E|)$).

Tarefa: Use o fato de que $P \subseteq NP$ para demonstrar que IS-BIPARTITE $\in NP$. Construa explicitamente o verificador e justifique.

Dica

Pela prova de $P \subseteq NP$, se $\pi \in P$, o verificador pode simplesmente executar o algoritmo que resolve π e ignorar o certificado.

Exercício 5 – IS-BIPARTITE: $P \subseteq NP$ (Resolução)

Passo 1 – O algoritmo existente em P

Seja A o algoritmo BFS que resolve IS-BIPARTITE em $O(|V| + |E|)$. A 2-colore o grafo: se uma contradição (dois vértices adjacentes com mesma cor) é encontrada, retorna NÃO; senão retorna SIM.

Passo 2 – Construção do verificador V

Dado (G, c) onde c é qualquer certificado:

$$\textit{Ignore } c \textit{ e execute } A(G).$$

Se $A(G) = \text{SIM} \Rightarrow V(G, c) = \text{SIM}$.
Se $A(G) = \text{NÃO} \Rightarrow V(G, c) = \text{NÃO}$.

Passo 3 – Verificação das propriedades

1. IS-BIPARTITE(G)=SIM $\Rightarrow$ $A(G)$=SIM $\Rightarrow$ $V(G, c)$=SIM. **OK.**
2. IS-BIPARTITE(G)=NÃO $\Rightarrow$ $A(G)$=NÃO $\Rightarrow$ $V(G, c)$=NÃO. **OK.**
3. V roda em $O(|V|+|E|)$ — polinomial. **OK.**

Conclusão: IS-BIPARTITE $\in P \Rightarrow$ IS-BIPARTITE $\in NP$, confirmando $P \subseteq NP$. ■

Exercício 6 – Classificar Problemas (Enunciado)

Enunciado

Para cada problema abaixo, indique: **P**, **NP** (provavelmente $\notin P$), ou **otimização** (não de decisão). Justifique.

a. **BUSCA-MAIOR-CLIQUE:** retornar o maior clique de G.
b. **EXISTE-CAMINHO:** há caminho de u a v em grafo não-ponderado?
c. **CLIQUE-K:** existe clique de tamanho $\geq k$ em G?
d. **3-SAT:** SAT com cláusulas de exatamente 3 literais.
e. **2-SAT:** SAT com cláusulas de exatamente 2 literais.

Dica: 2-SAT vs 3-SAT

NP é para problemas de decisão (SIM/NÃO). Otimização $\neq$ decisão.
2-SAT e 3-SAT têm tratabilidade diferente!

Exercício 6 – Classificar Problemas (Resolução)

(a) BUSCA-MAIOR-CLIQUE

Otimização — não é decisão. Reformulação: "existe clique $\geq k$?" $\in NP$.

(b) EXISTE-CAMINHO

Em P — BFS/DFS em $O(|V| + |E|)$. Também em NP por $P \subseteq NP$.

(c) CLIQUE-K

Em NP (certificado C; verificação $O(k^2)$). Provavelmente $\notin P$ — é NP-completo.

(d) 3-SAT

Em NP e NP-completo. Redução de SAT para 3-SAT mostra NP-completude.

(e) 2-SAT

Em P! Algoritmo via SCCs (Kosaraju/Tarjan) em $O(n + m)$. Restrição de 2 literais por cláusula torna o problema tratável.

Comparativo: P, NP e NP-Completo

Classe	Resolver	Verificar	Exemplos
P	Polinomial	Polinomial	IS-SORTED, BFS, MST, Palíndromo
NP	Desconhecido (talvez exp.)	Polinomial (com certif.)	SAT, SUDOKU, SOMA-SUBCONJ.
NP-Completo	Exponencial (acredita-se)	Polinomial (com certif.)	SAT, TSP, CLIQUE, CICLO-HAM.
Indecidível	Impossível	Impossível	Prob. da Parada

Hierarquia

$$P \subseteq NP \supseteq \text{NP-completo}$$

Se $P \neq NP$: NP-completo $\not\subseteq P$.

Distinção Fundamental: P vs NP na Prática

Como Distinguir na Prática

Ao encontrar um novo problema computacional:

1. **Tente encontrar algoritmo polinomial.** Se achar $\Rightarrow \pi \in P$.
2. **Tente reduzir de um NP-completo conhecido.** Se conseguir $\Rightarrow \pi$ é NP-completo.
3. **Mostre certificado e verificador polinomiais.** $\Rightarrow \pi \in NP$.

Casos Especiais que Mudam a Classe

- **SAT** $\in$ NP-completo; **2-SAT** $\in P$ (restrição de 2 literais por cláusula).
- **COLORAÇÃO** (geral) $\in$ NP-completo; **2-COLORAÇÃO** = IS-BIPARTITE $\in P$.
- **CICLO-HAMILTONIANO** (geral) $\in$ NP-completo; em grafos planos específicos pode ser P.

Mensagem Final

Fronteiras entre P e NP são sutis. Pequenas restrições podem tornar um problema NP-completo em um problema tratável.

Resumo – Aula 8 (Parte 1)

Conceitos Centrais

1. **Resolver vs Verificar:** verificar uma solução pronta geralmente é muito mais fácil que encontrá-la do zero.
2. **Certificado:** prova curta (tamanho polinomial) de que a resposta é SIM. Verificável em tempo polinomial.
3. **Classe NP:** problemas de decisão com certificado verificável em tempo polinomial.
4. $P \subseteq NP$**:** todo problema em P está em NP (provado).

Exemplos Vistos em NP

Problema	**Certificado**	**Custo Verif.**
SAT	Atribuição (n vars.)	$O(m \cdot n)$
SUDOKU	Grade 9×9 completa	$O(1)$
SOMA-SUBCONJ.	Lista V	$O(n)$
CICLO-HAM.	Seq. de vértices	$O(n)$
TSP	Tour	$O(n)$

Resumo – Aula 8 (Parte 2)

Mais Conceitos

1. **P vs NP:** questão aberta mais importante da CC; prêmio \$1M do Clay Institute.
2. **NP-Completo:** problemas "mais difíceis" de NP. Resolver qualquer um implicaria $P = NP$.
3. **Casos especiais:** uma pequena restrição pode tornar NP-completo em P (ex.: 3-SAT vs 2-SAT; k-COLORAÇÃO vs 2-COLORAÇÃO).

Intuição Final

- **Fácil de encontrar E verificar:** problema em P.
- **Fácil de verificar, difícil de encontrar:** problema em NP (provavelmente).
- **Se resolvermos UM NP-completo em polinomial:** todos em NP ficariam em P.

Reflexão

"P vs NP é a questão de saber se gênios matemáticos são fundamentalmente diferentes de estudantes aplicados, ou apenas mais rápidos." — Scott Aaronson

Próximas Aulas

O que Vem Depois

1. **Redução Polinomial:** técnica formal para demonstrar que um problema é tão difícil quanto outro.
2. **Provas de NP-Completude:** como reduzir SAT a novos problemas.
3. **Estratégias para Problemas NP-Difíceis:**
   - **Algoritmos de Aproximação:** solução ε-ótima em tempo polinomial.
   - **Heurísticas:** rápidas na prática (genéticos, simulated annealing, GRASP).
   - **Branch-and-Bound:** exato com poda inteligente.
   - **Parametrização:** polinomial em parâmetros fixos (FPT).

Leitura Recomendada

- Cormen et al., *Introduction to Algorithms*, Cap. 34 (NP-Completude).
- Sipser, *Introduction to the Theory of Computation*, Cap. 7.
- Garey & Johnson, *Computers and Intractability* (catálogo clássico).